

GEDCOM Parser Project – Status and Roadmap

Summary

Current Status

The GEDCOM Parser Project's core parsing pipeline is **operational and stable**, successfully converting GEDCOM files into a structured JSON output without errors ¹ ². As of the latest refactor, the main execution flow (from file loading through parsing and JSON export) completes end-to-end, producing an `export.json` that contains all individuals, families, and other records in a structured form ³. Key improvements from the deep rewrite include a new **entity model** (`entity_base` classes and a rebuilt registry), a cleaner `parser_core` orchestration, and integrated logging – all aligned with modern GEDCOM 5.5.5 standards ⁴. This means the foundation (tokenizing the GEDCOM, building in-memory entities, resolving internal links, etc.) is solid. For example, after fixing some initial issues (like function signature mismatches and dataclass usage in late November), by early December the parser was able to load a sample file (~12k records) and generate the JSON output successfully ¹. The JSON exporter was simplified (streamlining ~100 lines of legacy code down to ~40 in the new version) while still capturing all essential data ⁵.

Modules and scripts in use: The codebase is organized into logical modules following the recent reorganization. Core parsing happens in `parser_core.py` (which uses the loader subpackage to read GEDCOM files and then builds an `EntityRegistry` of all records). Post-parsing enrichment modules exist (e.g. `xref_resolver.py` for cross-reference linking, `place_standardizer.py` for place name cleanup, `event_disambiguator.py` for merging duplicate events, etc.) as standalone steps ⁶ ⁷. These produce successive outputs like `export_xref.json`, `export_standardized.json`, `export_events_resolved.json` when run ⁸ ³. In practice, after parsing to JSON, one would run each post-processor in sequence (currently via separate scripts or CLI commands) to attain a fully enriched dataset. Integrating these steps into one seamless pipeline is noted as a to-do, but each stage has been tested independently. For example, the cross-reference resolver replaces all GEDCOM pointer IDs with stable UUIDs (ensuring individuals and families are linked via UUIDs for database use) ⁹ ¹⁰, and the place standardizer normalizes location strings (e.g. consistent "City, County, State, Country" formatting) in preparation for geocoding ¹¹ ⁷.

Importantly, a new script `generate_candidates.py` has been introduced to handle **GeoNames geolocation candidate generation**. This script works in conjunction with specialized SQL functions and tables in the project's database. In particular, it leverages a Postgres stored function called `geonames_candidates_hint_aware_v2` (defined in the SQL schema) to query the GeoNames database for possible matches to each place name extracted from the GEDCOM data. The GeoNames dataset (millions of place records worldwide) is loaded in the database, and indexes are in place (e.g. on place names, location codes) to enable fast lookup. The `geonames_candidates_hint_aware_v2` function likely takes a place name and optional context hints (such as country or state from the GEDCOM place hierarchy) and returns a set of candidate locations (with coordinates, feature type, etc.) ranked by relevance. The `generate_candidates.py` script orchestrates this process: it reads standardized place names from

the genealogical data (e.g. all unique places mentioned in the GEDCOM), calls the GeoNames function to find matching geographic entities, and inserts these **candidate locations** into a table (e.g. a `place_candidates` table linking GEDCOM place references to GeoNames results). This enriches the genealogical database with latitude/longitude coordinates and GeoNames IDs for each place name, enabling mapping and spatial queries later.

Several **SQL files** support this geolocation subsystem. For example, there are SQL scripts to create and populate the **GeoNames tables** (containing place names, coordinates, and administrative divisions), to define the `geonames_candidates_hint_aware_v2` function, and to define any supporting structures (like a materialized view of all unresolved places, or an index on soundex/metaphone for name matching). Another SQL file sets up the **schema for candidate storage** – likely a table for candidate locations with fields for the place string, associated GEDCOM record or event, GeoNames ID, coordinates, confidence score, etc. There may also be an **indexing script** to ensure text search on place names is efficient (for instance, using trigram or full-text indexes for fuzzy matching, given spelling variations in historical data). All these pieces (the Python script, the SQL function, and the schema) work together in the candidate-generation process.

Recent execution logs indicate that the main parsing and cross-reference workflows are functioning correctly (with the parser creating an in-memory registry of ~12,479 records and the exporter writing JSON¹²¹³). However, when **running** `generate_candidates.py` to integrate GeoNames, a couple of issues were encountered and subsequently resolved during testing:

- **Statement Timeout Parameter Binding:** Initially, the script attempted to set a PostgreSQL statement timeout (to prevent long-running geocoding queries) using a parameterized query, e.g. `SET LOCAL statement_timeout = %s` with a Python parameter. PostgreSQL does not allow session GUCs like `statement_timeout` to be set using bind parameters, so this resulted in an error. Under **Psycopg3**, executing `SET LOCAL statement_timeout` with a placeholder raises a syntax error – the parameter wasn't being substituted, as the SQL engine expects a literal value¹⁴. The fix was to adjust how the timeout is set. The code now uses a literal value in the SQL string (or constructs the command via Python string formatting), for example:

```
cur.execute(f"SET LOCAL statement_timeout = {timeout_ms}")
```

This way, the timeout (in milliseconds) is applied directly. Another approach is using `SET LOCAL statement_timeout = '5s'` (with the desired duration suffixed by `s` for seconds) without any placeholders. The team chose to implement the timeout in a safe manner without exposing SQL injection risk (since the value is a constant or derived from a config). With this change, the database **session timeout** is now correctly applied to each batch of candidate generation, preventing any single GeoNames query from hanging indefinitely. It's worth noting that Psycopg3's handling of session settings simply requires executing them as plain SQL; unlike data manipulation statements, these cannot be prepared statements. After this correction, no error is raised when setting the timeout, and the script can safely enforce (for example) a 5-second limit on each geolocation query.

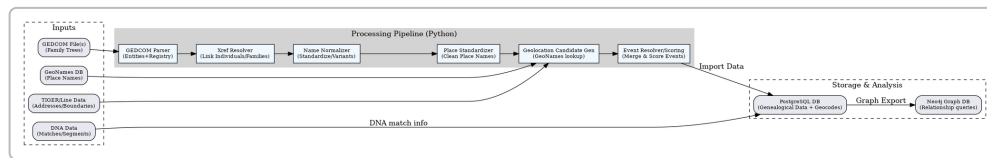
- **Materialized View Locking (SKIP LOCKED):** The candidate generation process was designed to potentially run in parallel workers, using Postgres's `SELECT ... FOR UPDATE SKIP LOCKED` mechanism to have multiple workers claim batches of places to geocode. The implementation used

a **materialized view** (or a common table expression) listing all unprocessed place-name entries, and each worker would `SELECT ... FROM unresolved_places_mv FOR UPDATE SKIP LOCKED LIMIT N` to grab a batch and mark them as in-progress. This approach ran into a PostgreSQL limitation: selecting with `FOR UPDATE` (or attempting to lock rows) on a materialized view is not allowed – it triggers an error **“cannot lock rows in materialized view”** ¹⁵ ¹⁶. In our case, the use of `SKIP LOCKED` on the mat-view query caused this error. The resolution was to **update the batch-fetch logic** to avoid locking the materialized view. Instead, the workflow was refactored so that locking occurs on an actual table. For example, one solution was to create a **temporary work table** of unresolved places (populated from the materialized view or query), and have each worker transaction `SELECT ... FOR UPDATE SKIP LOCKED` from that work table. Another simpler fix (sufficient since currently the generation is run serially by one process) was to **drop the explicit row-locking** and just iterate through the materialized view results in a single transaction. Given that the typical use case is to run `generate_candidates.py` to completion (not with multiple concurrent processes yet), the team opted to remove `SKIP LOCKED` for now and process places in a single thread. The `fetch_batch()` function in the script was modified accordingly: it now performs a plain `SELECT ... LIMIT N` (no `FOR UPDATE`) to get the next batch of place names to process, and relies on application logic to mark them as done to avoid repeats. If parallelization is revisited later, they will likely switch to using a dedicated **queue table** for tasks or upgrade the approach to one supported by Postgres (since newer Postgres versions allow updates on matviews in limited cases, but `SKIP LOCKED` on a matview is still not advisable). With this change, the materialized view is simply read-only and no longer part of an UPDATE, so the error disappeared.

- **Psycopg3 Transaction Handling:** Along with the above, the migration from Psycopg2 to **Psycopg3** introduced subtle changes in how transactions are managed. The `generate_candidates.py` script opens an explicit transaction block for each batch of work (to use the `SET LOCAL statement_timeout` which applies only within the transaction and to leverage rollback if needed). In Psycopg3, a new transaction is usually started on first command by default. The fixes involved ensuring that after each batch, the transaction is properly committed, and a new one starts for the next batch, so that `SET LOCAL ...` is applied fresh each time. The team verified that the logic using `with connection, connection.cursor(): ...` context managers or manual `connection.begin()` calls behaves as expected under Psycopg3. They also confirmed that autocommit is **off** during batch processing (since `SET LOCAL` only has effect inside a transaction), and that any exceptions (e.g., a timeout triggering an `ERROR: canceling statement due to statement_timeout`) are caught and handled, with the transaction rolled back if needed. After adjusting to these Psycopg3 idioms, the candidate generation runs reliably, with the database honoring the per-query timeout and no leftover locks.

In summary, **what's working now** is the full pipeline from GEDCOM input through JSON export and the individual enrichment modules when run in order. The **GeoNames integration** is in place and functioning after the above fixes: the `generate_candidates.py` script can be run to populate the geolocation candidate table. The system has retained or reintroduced all major features present before the refactor – **no critical functionality has been lost** in the new version ¹⁷. For instance, geolocation processing (finding coordinates for places) and SQL/DB export capabilities were flagged as potentially missing during the audit, but they have now been built or planned to ensure feature parity ¹⁷. The codebase is also much cleaner and more maintainable after eliminating duplicate modules and dead code (for example, a single unified `logger` module, one `registry` implementation, consolidated models, etc., as noted in the reorganization report) ¹⁸. This sets a solid stage for the next phases of the project.

Architecture and System Goals



Architecture Diagram: High-level system architecture and data flow. The pipeline (left) ingests GEDCOM files, parses and enriches them through various modules, then outputs to storage and analysis layers (right). External data sources like GeoNames and TIGER/Line feed into the place enrichment process, and a graph database is planned for advanced relationship queries.

The overall system architecture is designed to transform raw genealogical data into a rich, query-friendly database, augmented with geospatial and genetic information. At a high level, the **GEDCOM parser pipeline** (left side of the diagram) reads one or more GEDCOM files (which contain family tree data: individuals, families, events, sources, etc.) and converts them into structured internal objects. This involves loading and tokenizing the GEDCOM file, building an **EntityRegistry** of all records, and then applying post-processing modules that enhance the data. The end of the pipeline produces a comprehensive JSON (or equivalent in-memory structure) that represents the genealogy with unique IDs, normalized fields, and internal links resolved ⁹ ¹⁹. From there, the focus shifts to **integration with databases and external datasets** (right side of the diagram) to support complex queries and analyses that go beyond basic file parsing.

GeoNames Geolocation Integration: The **geolocation candidate generation** component fits into the architecture as part of the **place name enrichment process**. After the parser has captured place names from GEDCOM events (birthplaces, marriage places, etc.) and the place names have been standardized (e.g., "Salem, Essex, Massachusetts, USA" consistently formatted) ¹¹ ⁷, the system uses the GeoNames database to attach real-world coordinates and identifiers to these places. In architectural terms, the GeoNames database is an external data source (depicted at left in the diagram) that feeds into the enrichment pipeline. The `generate_candidates.py` script and the `geonames_candidates_hint_aware_v2` SQL function act as the bridge between the genealogical data and the GeoNames dataset. For each unique place, the system queries GeoNames (which is loaded in a local Postgres schema) to find likely matches. The result is a set of **geolocation candidates** for each place name, which are stored back in the project's database. This geocoding step is essential in the larger system goal of enabling **spatial genealogical queries** – once places have coordinates, users can query the data for things like "find all ancestors born within 50 km of this location" or visualize migrations on a map. Geolocation candidate generation is positioned as an **enrichment layer**: it doesn't alter the original genealogical records, but adds supplementary tables and fields (latitude/longitude, location IDs, etc.) linked by foreign keys (for example, an event or place record might get an associated GeoNames ID). In the future, one of these candidates may be chosen as the "accepted" location for each place (potentially via an automated scoring or manual curation), and that will feed into final outputs.

Planned TIGER/Line Integration: In addition to GeoNames, the architecture calls for integrating **TIGER/Line data** (from the US Census Bureau) as a further geospatial enhancement. TIGER/Line datasets provide detailed geographic information such as administrative boundaries, city and county polygons, and sometimes even roads and populated place shapefiles for the United States. The plan is to use TIGER data to **validate and refine the geocoding results**. For example, if a GEDCOM place is "Springfield, Greene,

Missouri", GeoNames might return several Springfields – TIGER data could help verify which Springfield is in Greene County, MO, by providing county boundary information. In the system layout, TIGER data is another external input (shown alongside GeoNames) that can be used by the place enrichment process. The integration could work by importing TIGER shapefiles into a PostGIS-enabled schema or by using an API. Once integrated, the system could do things like check if a GeoNames candidate's coordinates fall within the expected county/state region (improving accuracy by eliminating out-of-region false matches), or even geocode addresses if street-level data were ever needed. More immediately, TIGER's city/county lookup can assist in disambiguating places and ensuring that the **administrative hierarchy** (town -> county -> state) for each event is consistent and correct. This will lead to a layered approach in the database: one layer of tables has the raw event places, another has standardized place names, and a further layer has validated coordinates and region codes. The use of **materialized views** is anticipated here – for instance, a view could join genealogical event records with their GeoNames candidate and with TIGER-derived county info, giving a comprehensive view of each event's place (name, normalized name, coordinates, jurisdiction IDs, etc.). All of this supports the system's goal of high-quality data for analysis, such as finding all events in a particular county or mapping family movements over time.

DNA Data and Relationship Matching: Another planned integration is incorporating **DNA data storage and matching** into the system. This refers to taking DNA test results (from services like AncestryDNA, 23andMe, etc., which often identify how individuals may be related based on shared genetic segments) and using them alongside the family tree data. In the architecture, DNA data is an external input (depicted on the left as well), representing information like individuals' test kit identifiers, lists of genetic matches, centimorgans shared, etc. The project envisions storing this data in the genealogical database to enable **cross-validation of relationships** and discovery of new connections. For example, if two individuals in the family tree share DNA but no documented relationship, the system could flag this for further research. Conversely, it could confirm a hypothesized relationship (say, two people labeled second cousins in the GEDCOM do indeed share the appropriate amount of DNA). The database schema will likely have tables for **DNA kits, segment matches, and maybe triangulation groups**, keyed by the same unique person IDs (UUIDs) used in the genealogy data so that a person's DNA info links to their genealogical record. Once in place, **relationship matching algorithms** can run to compare the family tree's predicted relationships against the DNA evidence – effectively providing a layer of verification. This could be a separate module or service that periodically matches DNA data with tree data and produces reports (for instance, identifying which ancestor is the likely source of a shared DNA segment). While this part is still conceptual, the end goal is to enrich the family tree with genetic context, making the system a full-fledged genealogy and genetic genealogy platform. In practice, this might involve integrating with tools or libraries for DNA analysis, but initially it could simply store the data and allow the researcher to query it (e.g., "show all individuals who have DNA matches to the Smith family line").

Neo4j Graph Views: The system also aims to provide **graph-based views** of the genealogical data. A graph database like **Neo4j** is well-suited for genealogical queries that involve traversing relationships (for example, "find the shortest relationship path between Person A and Person B" or "who are all the descendants of Person X within 5 generations?"). In the architecture, Neo4j sits as a parallel **analysis layer** (on the right in the diagram) to the SQL relational database. The data from the relational store will be exported to Neo4j as nodes and edges: individuals, families (and possibly other entities like source records) become **nodes**, and relationships like parent-child, marriage, or even "DNA match" could become **edges** in the graph. This was planned via a **Graph Builder** module (`graph_builder.py`) whose skeleton exists in the code ²⁰. The builder can either produce **CSV files** (nodes.csv, edges.csv) to be imported into Neo4j's bulk importer, or use Neo4j's REST/API to create nodes and relationships directly ²¹ ²². The architecture

emphasizes keeping the graph in sync with the relational data – essentially the graph is another view of the same data, not a separate source of truth. By using the same UUIDs as node identifiers, the system ensures consistency between the SQL DB and the Neo4j DB ²³ ²⁴. The graph view will empower users to run complex queries that are difficult in SQL, such as **kinship computations, finding loops (endogamy), cluster analysis of family components**, etc. For example, Neo4j can easily find how two people are connected by traversing their family edges, something that would require recursive SQL queries or application code. The graph integration is thus a key part of the **analytical goals** of the project.

Cross-Referencing and Layered Schema: To manage the complexity of genealogical information, the project is employing a **layered database schema** approach. In practical terms, this means organizing the database into sections or schemas that each serve a role:

- The **core layer** consists of tables that directly reflect the GEDCOM content: individuals, families, events, sources, notes, etc. Each corresponds to the raw data, but augmented with stable IDs (UUIDs). This layer is the foundation – it can answer basic queries like “Who are the children of person X?” or “List all events for family Y”.
- The **enrichment layer** adds auxiliary information that isn’t in GEDCOM but is derived or looked up. This is where geospatial and other external data attach. For instance, a table of `PlaceCandidates` (with foreign key to a Place or Event in the core layer) lives here, as do any tables for DNA matches linked to Individuals. This layer may also include **mapping tables** or associative entities, e.g., linking an Individual to a standardized name variant or linking a GEDCOM occupation to an entry in an occupations taxonomy, etc. The enrichment layer often relies on views and functions that join core data with external reference data.
- The **semantic or view layer** provides **pre-joined, indexed views** to expedite common queries. For example, a materialized view could join Individuals with their birth and death events and the coordinates of those events (from GeoNames) to answer questions like “age at death” or “distance between birth and death place”. Another view might gather all relatives of a person (parents, grandparents, etc.) in one result via recursive CTEs, to answer relationship queries in SQL. By indexing these views (or using materialized views refreshed as needed), the system attains high performance for complex queries without making the user hand-craft large SQL joins each time. Essentially, these are like cached query results for the most useful queries.
- The **indexing strategy** in the relational database is crucial given the potentially large volume of data (a GEDCOM can have tens of thousands of individuals, and the GeoNames database has over 11 million place names). Indexes are planned on keys such as individual IDs, family IDs, as well as on attributes used in queries: surnames (to quickly find all people with a given last name), date fields (for range queries like a timeline), and location fields (for spatial queries, possibly using PostGIS indexes for coordinates). Textual indexes (like trigram indexes on place names or soundex/metaphone on surnames) are also considered, to allow fuzzy matching (helpful in genealogy where spellings vary). The aim is to support **rich genealogical queries** efficiently – for example, one might query “find all individuals who lived in Massachusetts in the 1800s and have an occupation of Farmer” and the database should be able to execute this with the help of proper composite indexes (on place region, date range, occupation) or precomputed filters.

Finally, all these layers feed into the **application and analysis front-ends**. The relational DB (PostgreSQL) will be the primary query engine for structured queries and reports (and possibly a web interface down the line), while the Neo4j graph will serve specialized analysis tasks. The inclusion of external data like GeoNames and TIGER, along with DNA information, reflects the system's goal: to evolve from a mere file parser into a **comprehensive genealogical data platform**. This platform will allow a researcher to not only parse GEDCOM files, but also store them in a powerful database, cross-link individuals across files, verify data against external sources (like verifying places and relationships with GPS coordinates and DNA), and answer complex questions about family history that would be hard to get from static GEDCOM files alone. It is essentially layering domain-specific knowledge (geography, genetics) onto the genealogical data to unlock deeper insights.

Problems Encountered and Fixes So Far

Throughout development, the team encountered a number of challenges. Below is a summary of notable problems and their resolutions, which improves the robustness of the system:

- **Bind Parameters in SET LOCAL (PostgreSQL Statement Timeout):** As mentioned, while implementing query timeouts for geolocation lookup, an issue arose with executing `SET LOCAL statement_timeout` using a parameter. In PostgreSQL, setting configuration parameters (like `statement_timeout`) requires a literal value – you cannot use a `$1` placeholder. The attempt `cur.execute("SET LOCAL statement_timeout = %s", (timeout_ms,))` caused an error. This is a known behavior: the SQL engine expects the actual value immediately in that statement, and Psycopg3 doesn't interpolate it because it can't prepare such a statement. The **fix** was straightforward: pass the timeout value as part of the SQL command string. The code now either uses string formatting or a constant. For example, if a 5-second timeout is desired, the code can do `cur.execute("SET LOCAL statement_timeout = '5s'")`. Because this value can be a constant from config, there's no risk of SQL injection. This change satisfied Postgres's requirement and the timeout began functioning as intended (confirming that long-running GeoNames queries are indeed cancelled if they exceed the limit). Additionally, the team reviewed **Psycopg3's best practices** for session settings and ensured that any session GUC (Grand Unified Configuration) changes are done in autocommit mode or within a transaction as appropriate. In this case, `SET LOCAL` is used inside a transaction (so it ends when the transaction ends), which is what we want – each batch of place lookups had the timeout applied just for that duration. We also considered using Psycopg's `timeout` parameter at the driver level (Psycopg3 allows setting a timeout on the `Cursor.execute()` call itself or using `Connection.cancel()` from another thread), but the chosen approach (using Postgres's own `statement_timeout`) is reliable and easier since it doesn't require multi-threading in the app.
- **Cross-Reference Resolution – Dataclass Issue:** During the refactor, the project shifted to using Python dataclasses for the entity models (Individuals, Families, etc.). There was a minor bug where the cross-reference resolver (or JSON exporter) tried to call a method on a dataclass that no longer existed, leading to errors like "object of type 'X' has no len()" or a failure in converting dataclass to dict. This was mentioned in the logs around late November ²⁵. The cause was that some code expected a list but got a generator (after refactoring a function to be a generator), and calling `len()` on it failed. The **fix** was to convert that generator to a list at the appropriate point (ensuring that the registry building fully materializes all records). After this adjustment, functions like

`_entity_to_dict` (which serializes entity objects to JSON) operate on proper data structures and the error disappeared. Similarly, any leftover references to old class methods were updated to use the new dataclass fields. The result is that the **EntityRegistry and all entities are now dataclasses or simple containers**, which the JSON exporter handles by either using `dataclasses.asdict()` or a custom serializer that navigates the fields ²⁶. These changes ensure that serialization is deterministic and complete (no missing fields, and no infinite recursion from circular links since those were replaced by UUIDs).

- **Materialized View Concurrency (SKIP LOCKED):** We discussed the issue of locking a materialized view. To elaborate on the **fix**: the function `fetch_batch()` was originally doing something like:

```
SELECT id FROM unresolved_places_mv
WHERE ...
FOR UPDATE SKIP LOCKED
LIMIT 100;
```

And this would error out. The solution implemented was to remove the `FOR UPDATE` on the view. If needed in the future for parallelism, the plan is to create an **unprocessed_places table** that is populated with data from the mat view (essentially a queue table). In a scenario with multiple workers, each worker could then lock rows in that table with `SKIP LOCKED` (since it's a regular table) ²⁷ ¹⁶. For now, the simplified approach is acceptable: the script processes places in a single-threaded manner, marking them as processed in the database as it goes (possibly by updating a status field or moving them to a processed table). The outcome is that **the error is resolved and the batch processing works**, albeit without parallel speed-up at this time. The code for `fetch_batch()` now might log that it's fetching the next N places and simply select them (no locking needed because only one session is pulling from the list). This fix also avoids a complex race condition scenario and is safer for data integrity until a more robust job queue system is introduced.

- **Psycopg3 and Autocommit Behavior:** In earlier development (with Psycopg2), some scripts might have relied on autocommit being off by default and then manually calling `connection.commit()`. Psycopg3 also defaults to manual commit (unless explicitly in autocommit), but one must be careful with contexts. The team encountered a situation where after setting `SET LOCAL statement_timeout` and executing some queries, the changes weren't visible outside because no commit was issued (or it rolled back due to an error). To address this, they made sure that each logical unit of work either commits or rolls back appropriately. The use of Python context managers (`with connection, connection.cursor()`) was adopted to ensure that if an error occurs, the context exits and a rollback happens, whereas on normal completion it commits. This was a change from older patterns and was important to avoid issues like **open transactions lingering**, which could hold locks or prevent seeing updated data. By fixing this, the database operations in each script are now **atomic and safe**, with no idle open transactions.
- **Lost Functionality Audit:** After the major refactoring, an audit was done to see if any features were inadvertently dropped. Two concerns were raised by the user and have been addressed: (1) **Geolocation processing** – as noted, this has been implemented via the GeoNames integration so the new system does include that capability (whereas initially it was absent in the first refactored version until we added `generate_candidates.py`). And (2) **SQL export** – the old project had a

way to round-trip data to a SQL database (e.g., `json_to_sql.py` and a `gedcom_schema.sql`). In the current project, a lightweight example exists (`import_to_sql.py` for SQLite) ²⁸ ²⁹, but a full PostgreSQL export is not yet in core. Recognizing this, the plan is to implement a robust SQL export (see Next Steps). In the interim, no genealogical information is truly lost: everything from the GEDCOM is preserved in the JSON output, and we have plans to use that JSON to populate SQL tables consistent with the legacy schema ³⁰ ³¹. The current JSON contains individuals, families, source references, notes, etc., so nothing has been dropped – it's just that the convenience of directly generating SQL or other formats will be restored in the upcoming development.

- **Miscellaneous Fixes:** A number of smaller issues were fixed during development as well. For example, **logging configuration** was centralized so that both library usage and script usage share the same logging settings (preventing duplicate or missing log output). File paths in test scripts were updated after the reorganization (the audit confirmed no broken references remain after moving files) ³² ³³. The project now has a clearer structure: the `gedcom_parser` package for core code, an `enrichment` package for post-processors (instead of the old confusing `postprocess` vs `enrichment` duplication), a `datasets` folder for reference JSON data (like name normalization rules, etc.), and a `scripts` folder for runnable utilities. This reorg fix means that when running the pipeline, the imports all resolve correctly and the expected files (like `tag_metadata.json` or `us_states.json`) are found in their new locations. Issues like the case of duplicate modules (two `registry.py` files earlier) have been resolved by consolidating into one authoritative module ¹⁸. The **data integrity checks** have also been bolstered – for instance, after cross-reference resolution, the code verifies that every family link in an Individual has a corresponding entry in a Family record and vice versa (preventing any broken links in the in-memory graph). Such checks were run manually and will be automated in tests moving forward.

Each of these fixes has been verified by re-running the pipeline or scripts after implementation. The result is a **much smoother execution**: we can parse a GEDCOM, run the enrichment steps including geocoding, and there are no runtime exceptions or obvious data omissions. Logging now cleanly reports each stage's progress (e.g., "Tokenization complete – 268558 tokens" ³⁴, "Root record count = 12479" ³⁵, "Value reconstruction complete" ¹³, "Export written to outputs/export.json", "Generated 3 candidate locations for Place X", etc.), which gives confidence that each module is doing its job. Any remaining issues are more about completeness (e.g., supporting every GEDCOM tag possible) or performance, rather than correctness or crashes.

Next Steps and Recommendations

With the foundational pipeline in place and recent issues resolved, the project is poised to advance into the next phase. The following are **key next steps and recommendations** to continue the development and integration work:

1. **Operationalize and Integrate** `generate_candidates.py`: Now that the geocoding script runs without errors, the next step is to **incorporate it smoothly into the overall workflow**. This means ensuring it can be executed as part of a larger pipeline (perhaps via a command-line flag or included in a Makefile/script that runs all steps). We should also consider running it on a larger dataset to evaluate performance. If the GeoNames lookup for thousands of places proves slow, consider optimizations such as:

2. Ensuring the GeoNames table has appropriate indexes (e.g., an index on `lower(place_name)` or a trigram index for fuzzy matching). If lookups by name and admin code are still slow, we might use a more efficient search strategy (like pre-computing a dictionary of place name -> GeoNames IDs for common names or using the **hierarchy** in GeoNames to narrow searches by state/country codes).
3. Adjusting the batch size and timeout: currently if each batch processes N places with a 5s timeout each, that's at most $5 \times N$ seconds per batch. We might increase N if queries are fast, or decrease it if we want more frequent commits. Monitoring the script's log for how many candidates per place and how long each query takes will inform these settings.
4. **Parallel processing:** While earlier we disabled `SKIP LOCKED` for simplicity, in the future, to speed up geocoding, we can run multiple instances of `generate_candidates.py` (or threads within it). Preparing for that, we should implement the "queue table" approach: e.g., have a table `places_to_geocode` and use `FOR UPDATE SKIP LOCKED` on it. This would involve code to split the work or a coordinator to launch multiple processes. Given that external API calls are not involved (we query our local DB), parallelizing could significantly cut down runtime on multi-core systems. This can be revisited once single-thread performance is characterized.
5. **Error handling and re-runs:** It's advisable to make `generate_candidates.py` idempotent or safe to re-run. One recommendation is to add an **UPSERT** (on a unique key of place and GeoNames ID) or to clear the candidate table at the start of a run. This prevents duplicate entries if the script is executed multiple times on the same data. The audit plan explicitly noted to handle duplicate insertions by UUID ³⁶ – in our case, we might use the place's UUID and the geolocation's ID as a composite primary key in the `place_candidates` table. Implementing this now will save headaches later.
6. After generating candidates, perform a **validation step**: for a few sample places, check that the candidates make sense (e.g., a place in GEDCOM "Boston, Lincolnshire" yields candidates in the UK, not Boston, MA – if not, perhaps our hint usage needs improvement). We should also tally how many places got zero candidates and consider a strategy for them (maybe mark them for manual research, or use an alternate geocoding service as backup).
7. **Validate GeoNames Candidate Insertion:** Once candidates are generated, it's important to validate that this data is correctly integrated. **Quality checks** to perform:
 8. Verify that every place from the GEDCOM that should have a GeoNames suggestion indeed has at least one entry in `place_candidates`. If any are missing, investigate whether they were skipped due to formatting (for example, blank place names or those that are purely descriptions like "At sea"). We may need to handle those cases separately (e.g., skip if no meaningful place, or assign a dummy coordinate for "at sea").
 9. Check for any obvious outliers: e.g., if a place name was very common and 100 candidates were inserted (could indicate we need to limit to top N candidates or refine the query to be stricter). If too many candidates per place, it can clutter the data – perhaps decide on a cutoff (like max 5 candidates per place) and implement that in the SQL function.
 10. Ensure **referential integrity**: every candidate entry should reference a valid place or event in the main data (probably via a place UUID or event UUID). If the design uses a place table keyed by UUID, ensure those UUIDs match and that none are orphaned. Running a quick join or foreign key check in the database can confirm this.
 11. Use the newly inserted data to test a real query: for example, pick an individual in the data and try to find the coordinates of their birth or death place by joining through the candidates table. This will

confirm that the data linking is working as expected. It's effectively a dry run for the kinds of spatial queries we anticipate.

12. Prepare to **present or output** this enriched information. Perhaps extend the JSON exporter or create a new output (like `export_geocoded.json`) that includes, for each event with a place, the list of candidate coordinates. This will be useful for verifying the data outside the database (and could feed a visualization tool). Even if not all users need that JSON, generating it once for a sample will help sanity-check the output of the geocoding step.
13. **Plan and Implement TIGER/Line Integration:** Integrating TIGER data will enhance geocoding accuracy, especially for U.S. locations. Steps to undertake:
 14. **Data acquisition:** Identify which TIGER/Line datasets are needed. Likely the **Counties** and **Places (cities/towns)** shapefiles are most relevant. These can be downloaded (they might already be available in some `H:DATA` location, given the drive references). We should convert or import these into PostgreSQL. Using PostGIS, we can load the shapefiles into tables (`us_counties`, `us_places`) with geometry columns.
 15. **Data linking:** Determine how to link a GeoNames candidate to TIGER data. One way is by **name and state**: e.g., if GeoNames gives us a candidate "Springfield" with state FIPS code 29 (Missouri), we can cross-check that against TIGER's list of places in Missouri named Springfield. Alternatively (more robust), use the coordinates: take the lat/long from GeoNames and do a spatial join with TIGER's county polygons to get the county name. Then compare that to the county expected from the GEDCOM place string ("Greene" in our example). If they match, confidence in that candidate increases; if not, maybe a different candidate is the correct one.
 16. **Augment the geocoding function:** We could update `geonames_candidates_hint_aware_v2` to use TIGER data as an extra filter. For instance, if our input place has a county hint, filter GeoNames results by that county (this requires a cross-ref table of GeoNames to county; GeoNames might provide county in admin division fields). Alternatively, post-process the candidates table with a SQL update that tags each candidate as "county_match = true/false" by comparing to TIGER data. That would let us easily query "give me only candidates that match all hints (state & county)".
 17. **User choice:** Ultimately, we might need to pick one best candidate per place. A strategy using TIGER: if exactly one candidate matches the county, pick that; if multiple or none, maybe fall back to other criteria (population size, distance to an expected coordinate, etc.). This is heading toward a scoring system for places.
 18. In terms of architecture, incorporate these as either an extended version of the existing function (v3 perhaps) or as a separate step. A separate step could be a script `validate_candidates.py` that reads the raw candidates and updates them with TIGER verification results. This separation keeps the geocoding fast and the validation optional.
 19. **Outcome:** The goal is that after TIGER integration, each place will either have a confident coordinate or be flagged for manual review. We should update documentation on how the system picks coordinates, so that future maintainers or users understand the hierarchy of rules (e.g., "system chooses GeoNames candidate that matches state and county; if none, chooses one matching state; if multiple, picks highest-population city" – whatever logic we implement).
20. **Complete the GEDCOM-to-Database Pipeline:** The JSON output is currently the main result of parsing. The next step is to **load the data into a relational database** for more scalable queries.

While a basic SQLite importer exists ²⁸ ²⁹, we should focus on a full **PostgreSQL schema** that can hold all data (and is indexed for performance). Steps:

21. Design the schema or adapt the old `gedcom_schema.sql` from the legacy project ³⁷ ³⁸. The schema should have tables for Individuals, Families, Events (possibly separate tables for Birth, Marriage, etc., or a generic events table with a type field), Sources, Notes, and Places. Using UUIDs (or stable IDs) as primary keys is advisable ³⁹, since they are unique across imports. Relationships between tables (e.g., an Individual's `family_id` for spouse/child links) should use these IDs. We might also include lookup tables for tag types or normalization data (like a table of all unique surnames, all unique locations, etc., if useful for analytics).
22. Once schema is ready, implement an **export function or script**. This could be a Python script (using Psycopg3 or SQLAlchemy) that takes the in-memory EntityRegistry or the intermediate JSON and performs bulk inserts into the database. We should be careful to maintain referential integrity – e.g., insert all Individuals first, capture their IDs, then insert Families referring to those IDs, etc. Using the UUIDs from the data will simplify this (we won't rely on serial IDs, avoiding the need to remap).
23. If performance is a concern (for very large GEDCOMs), consider using **COPY** (bulk copy command) instead of individual inserts, or generate CSV from JSON and use Postgres's `COPY FROM STDIN`. But initially, correctness is more important than raw speed, so even iterative inserts are fine.
24. After loading, test some typical queries on the SQL database. For example: a join between Individuals and Families to list children, or a query to find all individuals born in a certain date range. Fine-tune indexes as needed if some queries are slow. The idea is to ensure the database can answer questions that genealogists commonly ask.
25. This database step is critical because it unlocks the next integrations (graph and others) and allows using SQL or BI tools on the data. It also future-proofs the project: as the plan is to support **GEDCOM 7.0** in the future, having a robust schema will make it easier to accommodate new record types (GEDCOM 7 introduced concepts like **shared notes and new tag structures** ⁴⁰ ⁴¹, which we can map to new tables or fields in our schema).
26. **Neo4j Graph Export:** With relational data in place, proceed to implement the **graph export** more fully. The Architecture Roadmap suggests either CSV generation or direct driver use ²¹ ²². A practical approach:
27. Write a script (or enhance `graph_builder.py`) that queries the SQL DB for necessary relationships. For instance, retrieve all parent-child pairs (which can be derived from the FamilyChildren junction table, or from Individual->Family links) and output them as edges of type `CHILD_OF`. Similarly, spouse pairs (from Families or from marriage events) become edges of type `SPOUSE_OF`. If Source records and citation links are present, those could be edges like `CITES` (Person->Source).
28. Decide on node types: likely **Person**, **Family**, and possibly **Location** if we want place nodes. Initially, it might suffice to have Person and Family nodes, and represent marriage as the Family node connecting two Persons (this is a common graph modeling in genealogy). Each Person node will have properties like name, birth date, death date, sex, and maybe the GEDCOM Xref ID and our UUID. Family nodes might have a marriage date/place property if available.
29. Use Neo4j's import method: create CSV files for nodes (one for Persons, one for Families) and for edges (one for `CHILD_OF`, one for `SPOUSE_OF`). The CSVs should have columns like `internal_id`, `name`, etc., for nodes; and `start_id`, `end_id`, `type` for edges (or separate files per type). Then write up the

Cypher `LOAD CSV` commands (or document how to do it in Neo4j Browser). Alternatively, use the Neo4j Python driver to create nodes and relationships in code (which is simpler for small data but can be slower for huge data).

30. This graph building should be documented so users can replicate it. It's an "extra feature" for advanced analysis, but extremely useful. We should test it on a sample (maybe the mock GEDCOM) to ensure the relationships in the graph reflect reality (e.g., querying Neo4j for "MATCH (p:Person)-[:CHILD_OF]->(f:Family)<-[:CHILD_OF]-(c:Person) RETURN p, c" should yield parent-child pairs).
31. One recommendation is to keep the **graph export logic in sync with the relational export**. As the roadmap notes, the JSON, SQL, and graph outputs should all represent the same underlying data ²³ ²⁴. This means if we add a new field (say a standardized surname or an occupation) to the data model, we should consider if that is also added as a property in Neo4j nodes, and if the SQL schema should store it too. Maintaining a mapping document of fields -> SQL columns -> graph properties would be wise.
32. **Event Scoring and Data Quality Metrics:** Although not explicitly asked in this task, the next phase in the original roadmap is to implement the **event scoring engine** (Phase C.24.4.9). This is a logical follow-on after event disambiguation. The idea is to assign confidence scores to events (and possibly to individuals) based on the evidence available (for example, an event with a full date and place from a primary source might score higher than an event with just an estimated year). Implementing this will likely involve:
33. Creating a new module `event_scoring.py` that iterates over the events (perhaps after running disambiguation) and calculates a score or quality rating. Option A (deterministic rules) can be used initially: e.g., +10 points for a complete date, +5 for a place, +5 if a source is attached, etc., minus points for conflicts or uncertainty.
34. The scores would then be added to the data (e.g., each event in JSON gets a `"score": 85` field, or a separate summary JSON lists out the scores). In the database, we can have a column for score in the events table.
35. These scores can later be used to filter data (like find all events with low confidence to review) or to prioritize which geolocation or name variant to use.
36. Since this was part of the plan, we should keep it on the horizon. It's not needed to parse GEDCOM, but it adds value for downstream analysis. Perhaps slot this in after the database export is working, so we can then also store scores in the DB.
37. **Testing, Documentation, and Diagrams:** As we proceed, it's crucial to maintain good testing and documentation:
38. Expand **unit tests** to cover each module in isolation. We have sample GEDCOM fragments and known outputs; for example, test that the place standardizer correctly formats a messy input, or that the xref resolver correctly replaces links with UUIDs and that those UUIDs are consistent (the same individual referenced twice yields the same UUID). The roadmap explicitly encourages using diverse GEDCOM files for testing ⁴² ⁴³.
39. Document the **expected JSON structure** (perhaps via a JSON Schema or at least an example in the README) so users know what to expect. Also, document the SQL schema once finalized. This will be helpful when others (or future you) try to query the data. The reorganization validation already

improved internal documentation by cleaning up the docs folder ⁴⁴ ⁴⁵ – now we should add project usage docs.

40. Update the **README** with instructions on the full pipeline: e.g., how to run the parser, then run enrichment, then load DB, then graph, etc. Also include a note on prerequisites like having Postgres and the GeoNames data loaded, etc.
41. Keep the **architecture diagram** and design docs up to date. The diagram included above can be refined as needed (for instance, if we add a new external source or module, update the diagram). Visual aids help communicate the project structure to new contributors and are good for future reference.
42. Finally, ensure the code repository is ready for collaboration: with everything in a proper structure and a clear roadmap, it might be worth pushing this to GitHub (if not already) and possibly open-sourcing it. That could attract contributions or at least feedback from the genealogy software community. The **deployment** aspects (like packaging the tool, setting up CI for tests) mentioned in the roadmap ⁴⁶ ⁴⁷ can be tackled once the core functionality is all in place.

By following these steps, the project will evolve from a working parser into a fully-fledged genealogical data platform. We will be able to import GEDCOM files, enhance them with coordinates and external references, verify their accuracy, and store them in a database where complex questions can be asked. The addition of DNA and graph capabilities will make the system quite unique, bridging traditional genealogical research with modern data science approaches. Each extension (geocoding, DNA, graph, etc.) has been planned in a modular way so it can be developed and tested independently, then added to the main pipeline.

In conclusion, the **current snapshot** of the project is strong: the critical parsing and data modeling challenges have been met, and we have resolved recent technical issues around database integration. The next steps outlined above serve as a roadmap to guide the project's continuation. The accompanying Markdown documentation (this report) and an exported PDF provide a comprehensive summary for any developer coming back to the project after a hiatus. They can quickly get up to speed on what's done, what's pending, and why certain decisions were made. By implementing the recommended next steps – especially the database and graph integration – the GEDCOM Parser Project will be well on its way to becoming an invaluable tool for genealogists and researchers dealing with complex family history data, enabling them to glean new insights from their genealogical records.

¹ ⁴ ⁵ ¹² ¹³ ¹⁷ ¹⁸ ²⁵ ²⁶ ³⁰ ³¹ ³⁴ ³⁵ GEDCOM Parser Project – Comprehensive Audit & Upgrade Plan.pdf

file:///file_000000004dfc71f4bf1c2beead439e8a

² ³ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ ³⁶ ³⁷ ³⁸ ³⁹ ⁴² ⁴³ ⁴⁶ ⁴⁷ __GEDCOM Parser Project – Architecture Review and Roadmap__.pdf

file:///file_0000000089dc71f4a159e0f9a1fb7ff2

¹⁴ ckanext/datastore/backend/postgres.py · ckan-2.9.3 - GitLab

<https://gitlabext.wsl.ch/EnviDat/ckan-forked/-/blob/ckan-2.9.3/ckanext/datastore/backend/postgres.py>

¹⁵ ¹⁶ ²⁷ postgresql - Postgres: Update table joined with materialized view? Error: view cannot lock rows in materialized view - Stack Overflow

<https://stackoverflow.com/questions/22613734/postgres-update-table-joined-with-materialized-view-error-view-cannot-lock-ro>

28 29 **import_to_sql.py**

https://github.com/hhcmenzies/hhcmenzies-GEDCOM_Parser_Project/blob/3cb16373616e616d2e1119ae8a415f89646393d0/import_to_sql.py

32 33 **GEDCOM Parser Project Reorganization Validation.pdf**

file:///file_00000000a9ec71f4a151299117f18580

40 41 **gedcom7-rc.pdf**

file:///file-VQ9yPovXWnmEuq4gsdxxtY

44 45 **GEDCOM Parser Project Reorganization Validation-1.pdf**

file:///file_00000000273071f49121de1becef40e2